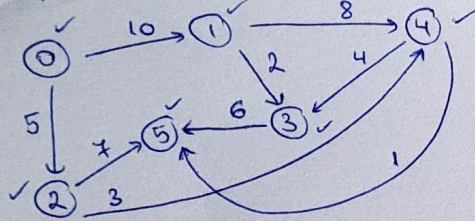


Topological Sorting for DAG (more orders)

(6, 9)

① graphical representation



② input file: (dagmorepaths.txt)

6 9

```
0 1 10
0 2 5
1 3 2
1 4 8
2 4 3
2 5 7
4 3 4
4 5 1
3 5 6
```

③ all topological orders:

✓ { 0, 1, 2, 4, 3, 5
 0, 2, 1, 4, 3, 5
 ~~0, 1, 4, 2, 3, 5~~
 ~~0, 1, 4, 3, 2, 5~~

invalid because 2 must
come before 4.

④ highest cost paths

Pair 1: 0 → 5.

path A: 0 → 1 → 3 → 5, cost: 10 + 2 + 6 = 18

path B: 0 → 1 → 4 → 3 → 5, cost: 10 + 8 + 4 + 6 = 28 (highest cost path)

path C: 0 → 1 → 4 → 5, cost: 10 + 8 + 1 = 19

path D: 0 → 2 → 5, cost: 5 + 7 = 12

path E: 0 → 2 → 4 → 3 → 5, cost: 5 + 3 + 4 + 6 = 18

path F: 0 → 2 → 4 → 5, cost: 5 + 3 + 1 = 9. (lowest cost path)

Pair 2: 0 → 3

path A: 0 → 1 → 3, cost: 10 + 2 = 12

path B: 0 → 1 → 4 → 3, cost: 10 + 8 + 4 + ~~4~~ = 22 (highest cost path)

path C: 0 → 2 → 4 → 3, cost: 5 + 3 + 4 = 12

Topological sort ensures each vertex is processed only after all its predecessors are finished. In a DAG, this allows calculating the highest cost by "relaxing" ~~the~~ incoming edges once, building the solution incrementally via DP in $O(n+m)$ time.

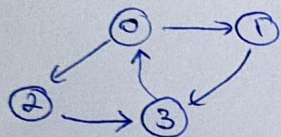
Cycle Case:

(4, 5)

input file:

```

4 5
0 1 0
1 3 0
3 0 0
2 3 0
0 2 0
    
```



calls	x, y	inProcess	fullyProcessed	sorted	OK
initialization		{ }	{ }	[]	
TopoSortDFS(G, 0, [], { }, { })	x = 0 y = 3	{ } { 0 }	{ }	[]	
TopoSortDFS(G, 3, [], { }, { 0 })	x = 3 y = 1	{ 0 } { 0, 3 }	{ }	[]	
TopoSortDFS(G, 1, [], { }, { 0, 3 })	x = 1 y = 0	{ 0, 3 }	{ }	[]	
		y = 0 ∈ inProcess ⇒			false
	y = 2				
TopoSortDFS(G, 2, [], { }, { 0, 3 })	x = 2 y = 0	{ 0, 3 }			
		y = 0 ∈ inProcess ⇒			false

∴ G is not a DAG.